

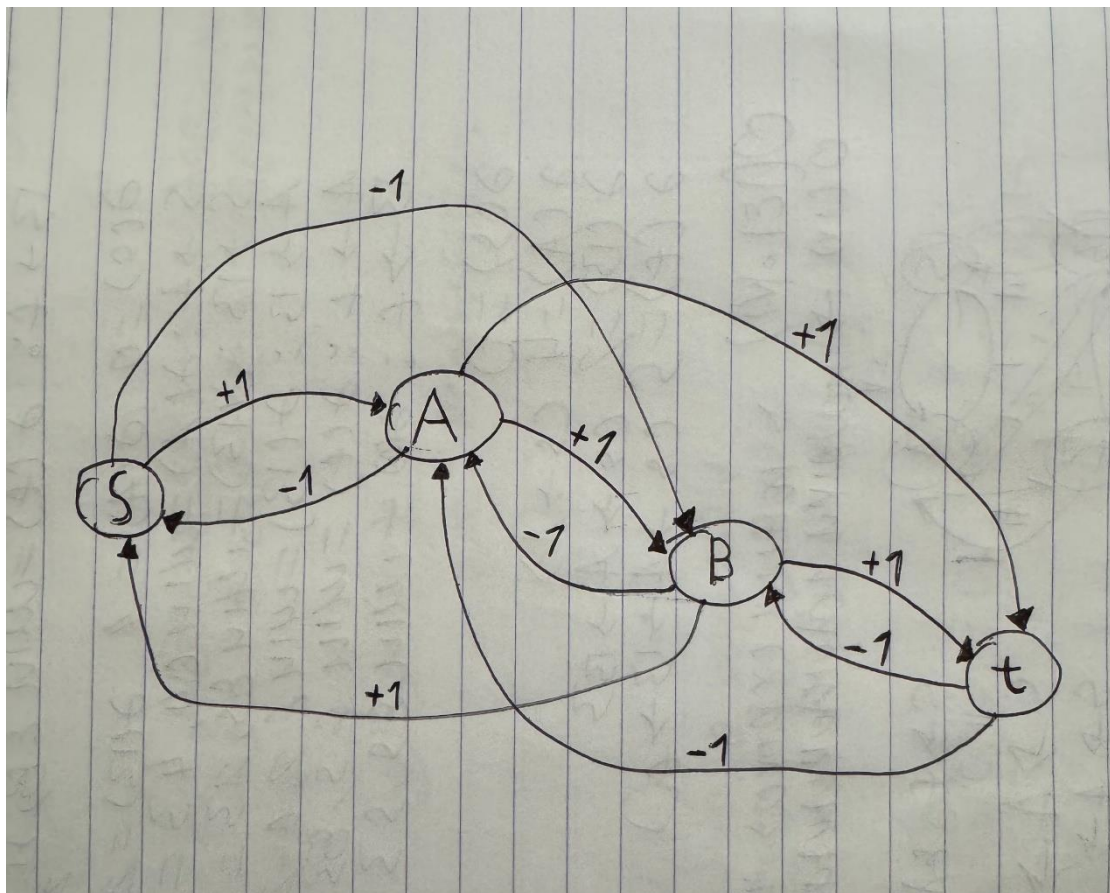
Άσκηση: Συντομότερες Διαδρομές με Θετικά και Αρνητικά Βάρη Κόστος .Καυσίμου σε Περιοχή με Υψομετρικές Διαφορές

Μέλη Ομάδας για την Εργασία στο μάθημα Αλγόριθμοι και Πολυπλοκότητα:

Νικόλαος Κατσικαβάλης, AM:02618, email:nkatsikavalis@uth.gr

Γεώργιος Παπαμικρούλης, AM:02675, email:gparamikr@uth.gr

Ερώτημα 1:



Οι συνδέσεις είναι:

- $s \rightarrow A$ έχει βάρος $+1$, $A \rightarrow s$ έχει βάρος -1
- $s \rightarrow B$ έχει βάρος -1 , $B \rightarrow s$ έχει βάρος $+1$
- $A \rightarrow B$ έχει βάρος $+1$, $B \rightarrow A$ έχει βάρος -1
- $A \rightarrow t$ έχει βάρος $+1$, $t \rightarrow A$ έχει βάρος -1
- $B \rightarrow t$ έχει βάρος $+1$, $t \rightarrow B$ έχει βάρος -1

Ερώτημα 2:

Οι ακμές που έχουν αρνητικό πρόσημο (βάρος) είναι συνολικά πέντε:

- Η ακμή (s,B)
- Η ακμή (A,s)
- Η ακμή (B,A)
- Η ακμή (t,A)
- Η ακμή (t,B)

Ερώτημα 3:

Στο πλαίσιο του προβλήματος, το βάρος της ακμής αντιπροσωπεύει το καθαρό κόστος καυσίμου. Όταν ένα όχημα κινείται σε ανηφόρα (προς υψηλότερο υψόμετρο), ο κινητήρας πρέπει να παράγει έργο ενάντια στη βαρύτητα, άρα καταναλώνει καύσιμο (+1). Αντίθετα, όταν κινείται σε κατηφόρα (προς χαμηλότερο υψόμετρο), η βαρύτητα υποβοηθά την κίνηση. Το όχημα μπορεί να ρολάρει, μειώνοντας έτσι τη συνολική δαπάνη. Συνεπώς, το καθαρό κόστος θεωρείται αρνητικό (-1). Στους αλγόριθμους συντομότερης διαδρομής, το αρνητικό βάρος λειτουργεί ως "ανταμοιβή", μειώνοντας το συνολικό μήκος του μονοπατιού.

Ερώτημα 4:

Όχι, δεν επιτρέπεται (δεν εγγυάται το σωστό αποτέλεσμα). Ο αλγόριθμος του Dijkstra έχει μια θεμελιώδη προϋπόθεση για να λειτουργήσει σωστά: όλα τα βάρη των ακμών του γράφου πρέπει να είναι μη αρνητικά ($w(u, v) \geq 0$). Ο αλγόριθμος λειτουργεί με μια στρατηγική απληστίας (greedy). Όταν εξάγει έναν κόμβο u από την ουρά προτεραιότητας, θεωρεί ότι έχει ήδη βρει την οριστικά συντομότερη διαδρομή προς τον u . Αν υπάρχουν αρνητικά βάρη, θα μπορούσε αργότερα να ανακαλυφθεί μια άλλη διαδρομή προς τον u , μέσω ενός άλλου κόμβου, που χάρη σε μια ακμή με αρνητικό βάρος να προσφέρει μικρότερο συνολικό κόστος. Ο Dijkstra δεν επανεξετάζει κόμβους που έχουν ήδη εξορυχθεί, οπότε θα αποτύγχανε να εντοπίσει αυτή τη φθηνότερη διαδρομή.

Σε αυτόν τον γράφο υπάρχουν αρκετές ακμές με αρνητικό βάρος (-1), οπότε ο Dijkstra δεν είναι ο κατάλληλος αλγόριθμος (θα έπρεπε να χρησιμοποιηθεί ο Bellman-Ford).

Ερώτημα 5:

Αρχικοποίηση:

- $\text{dist}(s) = 0$
- $\text{dist}(A) = \text{άπειρο}$, $\text{dist}(B) = \text{άπειρο}$, $\text{dist}(t) = \text{άπειρο}$
- Ουρά $Q = \{s(0), A(\text{άπειρο}), B(\text{άπειρο}), t(\text{άπειρο})\}$

Βήμα 1:

- Εξάγουμε τον κόμβο με την ελάχιστη απόσταση. Είναι ο s (με $\text{dist} = 0$). Οριστικοποιούμε το $\text{dist}(s) = 0$.
- Εξετάζουμε τους γείτονες του s : τον A και τον B .
 - Ακμή $s \rightarrow A$: $w(s,A) = +1$. Νέα απόσταση $= \text{dist}(s) + 1 = 0 + 1 = 1$. Αφού $1 < \text{άπειρο}$, ενημερώνουμε: $\text{dist}(A) = 1$.
 - Ακμή $s \rightarrow B$: $w(s,B) = -1$. Νέα απόσταση $= \text{dist}(s) - 1 = 0 - 1 = -1$. Αφού $-1 < \text{άπειρο}$, ενημερώνουμε: $\text{dist}(B) = -1$.
- $Q = \{B(-1), A(1), t(\text{άπειρο})\}$

Βήμα 2:

- Εξάγουμε τον κόμβο με την ελάχιστη απόσταση. Είναι ο B (με $\text{dist} = -1$). Οριστικοποιούμε το $\text{dist}(B) = -1$.
- Εξετάζουμε τους γείτονες του B : τον s , τον A και τον t . (Ο s έχει ήδη εξαχθεί, αλλά ας δούμε την πράξη: $\text{dist}(B) + w(B,s) = -1 + 1 = 0$, που δεν βελτιώνει το $\text{dist}(s)=0$).
 1. Ακμή $B \rightarrow A$: $w(B,A) = -1$. Νέα απόσταση $= \text{dist}(B) + w(B,A) = -1 - 1 = -2$. Αφού $-2 < \text{dist}(A)=1$, η απόσταση του A ενημερώνεται σε: $\text{dist}(A) = -2$
 2. Ακμή $B \rightarrow t$: $w(B,t) = +1$. Νέα απόσταση $= \text{dist}(B) + w(B,t) = -1 + 1 = 0$. Αφού $0 < \text{άπειρο}$, ενημερώνουμε: $\text{dist}(t) = 0$.

- $Q = \{A(-2), t(0)\}$

Βήμα 3:

- Εξάγουμε τον A (με $\text{dist} = -2$). Οριστικοποιούμε το $\text{dist}(A) = -2$.
- Εξετάζουμε τους γείτονες του A : τον s , τον B και τον t .
 - Ακμή $A \rightarrow s$: -1 . Νέα απόσταση: $-2 - 1 = -3$. Εδώ το $\text{dist}(s)$ θα γινόταν -3 .
 - Ακμή $A \rightarrow B$: $+1$. Νέα απόσταση: $-2 + 1 = -1$.
 - Ακμή $A \rightarrow t$: $+1$. Νέα απόσταση: $-2 + 1 = -1$. Αφού $-1 < \text{dist}(t)=0$, η απόσταση του t ενημερώνεται σε: $\text{dist}(t) = -1$.
- $Q = \{t(-1)\}$

Βήμα 4:

- Εξάγουμε τον t (με $\text{dist} = -1$). Οριστικοποιούμε το $\text{dist}(t) = -1$.
- Γείτονες του t : A και B .

- 1) Ακμή $t \rightarrow A$: -1. Νέα απόσταση: $-1 - 1 = -2$.
- 2) Ακμή $t \rightarrow B$: -1. Νέα απόσταση: $-1 - 1 = -2$. Προσοχή: Το $\text{dist}(B)$ ήταν -1. Τώρα βρήκαμε διαδρομή με -2! Αλλά ο B έχει ήδη εξαχθεί. Στην κλασική υλοποίηση Dijkstra, αυτή η βελτίωση αγνοείται. Αν δεν αγνοηθεί, μπαίνουμε σε ατέρμονο βρόχο.

Αποτελέσματα Dijkstra

- $\text{dist}(s) = 0$
- $\text{dist}(A) = -2$
- $\text{dist}(B) = -1$
- $\text{dist}(t) = -1$

Ερώτημα 6:

- Από $s \rightarrow B$ κοστίζει -1.
- Από $B \rightarrow A$ κοστίζει -1.
- Από $A \rightarrow s$ κοστίζει -1.

Υπάρχει ένας κύκλος: $s \rightarrow B \rightarrow A \rightarrow s$

Το κόστος αυτού του κύκλου είναι: $w(s,B) + w(B,A) + w(A,s) = (-1) + (-1) + (-1) = -3$. Υπάρχει αρνητικός κύκλος.

Λόγω αυτού του κύκλου, μπορούμε να κάνουμε όσους γύρους θέλουμε στο μονοπάτι $s \rightarrow B \rightarrow A \rightarrow s$, μειώνοντας το κόστος κατά 3 σε κάθε γύρο.

Επομένως, το ελάχιστο κόστος από το s προς οποιονδήποτε κόμβο που είναι προσβάσιμος από αυτόν τον κύκλο (δηλαδή τους s , A , B , και κατ' επέκταση και τον t μέσω της ακμής $A \rightarrow t$ ή $B \rightarrow t$) δεν ορίζεται, ή τείνει στο μείον άπειρο.

Άρα, οι πραγματικές συντομότερες αποστάσεις είναι -άπειρο για όλες τις κορυφές.

Ερώτημα 7:

Ο αλγόριθμος Dijkstra έδωσε πεπερασμένες (αλλά λανθασμένες) αποστάσεις: $\text{dist}(A)=-2$, $\text{dist}(B)=-1$, $\text{dist}(t)=-1$. Οι πραγματικές αποστάσεις, ωστόσο, τείνουν στο μείον άπειρο λόγω της ύπαρξης του αρνητικού κύκλου $s \rightarrow B \rightarrow A \rightarrow s$. Ο Dijkstra διαφωνεί πλήρως με την πραγματικότητα, καθώς δεν έχει μηχανισμό για να ανιχνεύσει αρνητικούς κύκλους ούτε να αναθεωρήσει αποστάσεις κόμβων που έχουν ήδη κλείσει.

Ερώτημα 8:

Όπως βρήκαμε στο ερώτημα 6, ναι, υπάρχει αρνητικός κύκλος: ο $s \rightarrow B \rightarrow A \rightarrow s$ με συνολικό βάρος -3. Το πρόβλημα των συντομότερων διαδρομών δεν έχει λύση (είναι ill-posed) για τα μονοπάτια που περνούν από αυτόν τον κύκλο. Στο συγκεκριμένο πλαίσιο, αν το βάρος είναι το κόστος καυσίμου, ένας αρνητικός κύκλος σημαίνει ότι το όχημα μπορεί να κάνει κυκλικές διαδρομές και να παράγει καύσιμο επ' άπειρον, κάτι που πρακτικά είναι αδύνατο και φυσικά δεν έχει φυσικό νόημα. Μαθηματικά, το κόστος μπορεί να γίνεται αυθαίρετα μικρό, οπότε δεν υπάρχει συντομότερη διαδρομή.

Ερώτημα 9:

Η συνθήκη $w(u,v) = -w(v,u)$ είναι απλώς ένας κανόνας αντισυμμετρίας. Αυτό δεν εγγυάται την απουσία αρνητικών βαρών, ούτε την απουσία αρνητικών κύκλων.

Ο Dijkstra αποτυγχάνει εξαιτίας της ίδιας της ύπαρξης αρνητικών βαρών, επειδή σπάει η θεμελιώδης υπόθεσή του ότι προσθέτοντας μια ακμή σε ένα μονοπάτι, το συνολικό κόστος δεν μπορεί ποτέ να μειωθεί. Αν το συνολικό κόστος μπορεί να μειωθεί (λόγω κατηφόρας), ο Dijkstra δεν θα εξετάσει ξανά μονοπάτια που απέρριψε νωρίτερα, με αποτέλεσμα να βγάλει λάθος συμπεράσματα. Επιπλέον, όπως είδαμε, αυτή η συνθήκη επέτρεψε τον σχηματισμό ενός αρνητικού κύκλου τριών κόμβων, ο οποίος καταστρέφει εντελώς την έννοια της συντομότερης διαδρομής, κάτι που ο Dijkstra ούτως ή άλλως δεν μπορεί να χειριστεί.

ΕΠΕΚΤΑΣΗ:

```
import random
```

```
import heapq
```

```
import itertools
```

```
def generate_random_graph(nodes, p=0.7, q=0.3):
```

```
    """
```

Δημιουργεί έναν τυχαίο γράφο σύμφωνα με τις πιθανότητες της άσκησης.

Επιστρέφει τη δομή του γράφου (λίστα γειτνίασης) και ένα boolean

για το αν υπάρχουν αρνητικές ακμές.

```
    """
```

```
    adj = {node: [] for node in nodes}
```

```
    has_negative_edge = False
```

```

# Βρίσκουμε όλα τα πιθανά ζεύγη κορυφών

pairs = list(itertools.combinations(nodes, 2))

for u, v in pairs:

    # Πιθανότητα p=0.7 να υπάρχει σύνδεση μεταξύ των δύο κορυφών

    if random.random() < p:

        has_negative_edge = True # Εφόσον τα βάρη είναι 1 και -1, σίγουρα θα
        υπάρχει αρνητική ακμή

        # Πιθανότητα q=0.3 η πρώτη κατεύθυνση να είναι -1 και η αντίθετη +1

        if random.random() < q:

            weight_uv, weight_vu = -1, 1

        else:

            weight_uv, weight_vu = 1, -1

        adj[u].append((v, weight_uv))

        adj[v].append((u, weight_vu))

return adj, has_negative_edge

```

```

def dijkstra(graph, start):

```

```

    """

```

Κλασική υλοποίηση Dijkstra.

Σημείωση: Αποφεύγουμε την επανεξέταση επισκεπτόμενων κόμβων

για να μην πέσει σε ατέρμονο βρόχο λόγω αρνητικών κύκλων.

```

    """

```

```

    distances = {node: float('inf') for node in graph}

```

```

    distances[start] = 0

```

```

pq = [(0, start)]

visited = set()

while pq:

    current_dist, u = heapq.heappop(pq)

    if u in visited:

        continue

    visited.add(u)

    for v, weight in graph[u]:

        if current_dist + weight < distances[v]:

            distances[v] = current_dist + weight

            heapq.heappush(pq, (distances[v], v))

return distances

def bellman_ford(graph, nodes, start):

    """

    Υλοποίηση Bellman-Ford που διαχειρίζεται αρνητικά βάρη

    και ανιχνεύει αρνητικούς κύκλους.

    """

    distances = {node: float('inf') for node in nodes}

    distances[start] = 0

    # Χαλάρωση (relaxation) ακμών |V| - 1 φορές

    for _ in range(len(nodes) - 1):

        for u in nodes:

```

```

    for v, weight in graph[u]:

        if distances[u] != float('inf') and distances[u] + weight < distances[v]:

            distances[v] = distances[u] + weight

# Έλεγχος για αρνητικό κύκλο στο (V)-οστο βήμα

has_negative_cycle = False

for u in nodes:

    for v, weight in graph[u]:

        if distances[u] != float('inf') and distances[u] + weight < distances[v]:

            has_negative_cycle = True

            break

    if has_negative_cycle:

        break

    return distances, has_negative_cycle

def run_simulation(num_trials=10000):

    nodes = ['s', 'A', 'B', 't']

    stats = {

        'total_graphs': num_trials,

        'with_negative_edges': 0,

        'with_negative_cycles': 0,

        'dijkstra_correct': 0,

        'dijkstra_incorrect': 0

    }

```



```

print(f'Εκτέλεση προσομοίωσης για {num_trials} τυχαίους γράφους...\n")

for _ in range(num_trials):

    # 1. Δημιουργία Γράφου

    graph, has_neg_edges = generate_random_graph(nodes, p=0.7, q=0.3)

    if has_neg_edges:

        stats['with_negative_edges'] += 1

        # 2. Εκτέλεση Dijkstra

        dijkstra_dists = dijkstra(graph, 's')

        # 3. Εκτέλεση Bellman-Ford

        bf_dists, has_neg_cycle = bellman_ford(graph, nodes, 's')

        # 4. Καταγραφή & Σύγκριση Αποτελεσμάτων

        if has_neg_cycle:

            stats['with_negative_cycles'] += 1

            # Αν υπάρχει αρνητικός κύκλος, ο Dijkstra εξ' ορισμού δίνει λάθος

            # αποτέλεσμα (βγάζει πεπερασμένη απόσταση ενώ είναι -άπειρο)

            stats['dijkstra_incorrect'] += 1

        else:

            # Αν ΔΕΝ υπάρχει αρνητικός κύκλος, ελέγχουμε αν οι αποστάσεις

            ταυτίζονται

            if dijkstra_dists == bf_dists:

                stats['dijkstra_correct'] += 1

            else:

                stats['dijkstra_incorrect'] += 1

```

```

# Εμφάνιση Αποτελεσμάτων

print("=" * 50)

print(" ΣΤΑΤΙΣΤΙΚΑ ΑΠΟΤΕΛΕΣΜΑΤΑ ΠΡΟΣΟΜΟΙΩΣΗΣ")

print("=" * 50)

print(f"Συνολικοί Γράφοι που ελέγχθηκαν    : {stats['total_graphs']}")

print(f"Γράφοι με Αρνητικές Ακμές          : {stats['with_negative_edges']}")

print(f"Γράφοι με Αρνητικούς Κύκλους       : {stats['with_negative_cycles']}")

print("-" * 50)

print(f"Περιπτώσεις που ο Dijkstra ήταν ΣΩΣΤΟΣ : {stats['dijkstra_correct']} "

      f"({(stats['dijkstra_correct']/num_trials)*100:.2f}%)")

print(f"Περιπτώσεις που ο Dijkstra ήταν ΛΑΘΟΣ  : {stats['dijkstra_incorrect']} "

      f"({(stats['dijkstra_incorrect']/num_trials)*100:.2f}%)")

print("=" * 50)


if __name__ == "__main__":

    run_simulation(10000)

```

TERMINAL

Εκτέλεση προσομοίωσης για 10000 τυχαίους γράφους...

```

=====

ΣΤΑΤΙΣΤΙΚΑ ΑΠΟΤΕΛΕΣΜΑΤΑ ΠΡΟΣΟΜΟΙΩΣΗΣ

=====

Συνολικοί Γράφοι που ελέγχθηκαν    : 10000
Γράφοι με Αρνητικές Ακμές          : 9996
Γράφοι με Αρνητικούς Κύκλους       : 7470
-----

```

Περιπτώσεις που ο Dijkstra ήταν ΣΩΣΤΟΣ : 2530 (25.30%)

Περιπτώσεις που ο Dijkstra ήταν ΛΑΘΟΣ : 7470 (74.70%)

Ανάλυση Πειραματικών Δεδομένων & Συμπεράσματα Συχνότητα

1)Εμφάνισης Αρνητικών Ακμών: Η παρουσία αρνητικών ακμών στο 99,96% των περιπτώσεων (9.996 από τους 10.000 γράφους) εξηγείται άμεσα από τους κανόνες κατασκευής. Καθώς κάθε σύνδεση μεταξύ δύο κορυφών δημιουργεί υποχρεωτικά ένα ζεύγος αντίθετων ακμών με βάρη +1 και -1, η πιθανότητα ένας γράφος να μην διαθέτει καμία αρνητική ακμή είναι πρακτικά μηδαμινή, απαιτώντας την πλήρη απουσία συνδέσεων στο δίκτυο .

2)Σχηματισμός Αρνητικών Κύκλων: Η εμφάνιση αρνητικών κύκλων σε ποσοστό 74,7% οφείλεται στην υψηλή πυκνότητα του γράφου ($p=0,7$) και στην τυχαία κατανομή των βαρών. Λόγω του μικρού αριθμού κορυφών, η πιθανότητα να σχηματιστεί ένα κλειστό μονοπάτι με αρνητικό συνολικό άθροισμα είναι ιδιαίτερα υψηλή. Στο πλαίσιο του προβλήματος, η ύπαρξη τέτοιων κύκλων καθιστά τη διαδρομή ενεργειακά παράδοξη, καθώς η επ' άπειρον περιφορά σε αυτούς θα οδηγούσε σε απεριόριστη παραγωγή καυσίμου, γεγονός που στερείται φυσικού νοήματος.

3)Αποτυχία του Αλγορίθμου Dijkstra: Παρατηρείται απόλυτη ταύτιση μεταξύ του αριθμού των αρνητικών κύκλων (7.470) και των αποτυχιών του Dijkstra. Η «άπληστη» φύση του αλγορίθμου τον οδηγεί στον εγκλωβισμό σε τοπικά βέλτιστες λύσεις, αδυνατώντας να αναγνωρίσει ότι η απόσταση προς μια κορυφή μπορεί να μειώνεται διαρκώς λόγω ενός αρνητικού κύκλου. Στις περιπτώσεις αυτές, ο Dijkstra εξάγει πεπερασμένες τιμές, ενώ στην πραγματικότητα η συντομότερη διαδρομή δεν ορίζεται (τείνει στο -άπειρο) .

4)Περιπτώσεις Ορθότητας: Το ποσοστό επιτυχίας του αλγορίθμου (25,3%) περιορίζεται αποκλειστικά στους γράφους όπου δεν σχηματίστηκε αρνητικός κύκλος. Αυτό αποδεικνύει ότι, ενώ η παρουσία μεμονωμένων αρνητικών ακμών μπορεί υπό συνθήκες να μην οδηγήσει σε σφάλμα, η εμφάνιση έστω και ενός αρνητικού κύκλου καθιστά τον Dijkstra εντελώς αναξιόπιστο.

Συνολικό Συμπέρασμα: Τα πειραματικά δεδομένα υπογραμμίζουν ότι η συνθήκη $w(u,v) = -w(v,u)$ δεν διασφαλίζει την ορθή λειτουργία του Dijkstra . Για την ασφαλή εύρεση συντομότερων διαδρομών σε δίκτυα με υπομετρικές διαφορές που επιτρέπουν κατηφορικές κινήσεις (αρνητικά βάρη), είναι απαραίτητη η χρήση αλγορίθμων που ανιχνεύουν αρνητικούς κύκλους, όπως ο Bellman-Ford, καθώς η πιθανότητα εμφάνισης υπολογιστικών παθολογιών είναι εξαιρετικά υψηλή.